3.1) توی Batch GD مقدار loss خیلی آروم و تقریبا خطی کم میشه. این به خاطر اینه که تو هر مرحله از کل داده‌ها برای محاسبه گرادیان استفاده میشه، بنابراین آپدیت‌ها خیلی محافظه‌کارانه و بدون نوسان هستن، ولی در عوض سرعت یادگیری پایین‌تره.

از اون طرف، SGD خیلی سریع‌تر loss رو کم می‌کنه. هرچند نویز بیشتری داره چون با نمونه‌های تصادفی کار میکنه، ولی همین نویز باعث می‌شه مدل زودتر به سمت مقادیر بهتر حرکت کنه و توی چند epoch اول پیشرفت زیادی داشته باشه. توی این نمودار هم می‌بینیم که SGD خیلی زود از Batch GD جلو می‌زنه و به loss کمتری می‌رسه.

در مجموع می‌شه گفت Batch GD پایدارتره ولی کندتر یاد می‌گیره، در حالی که SGD سریع‌تر همگرا می‌شه و برای آموزش عملی، مخصوصا وقتی زمان مهمه، انتخاب مناسب‌تریه.

3.2) از روی نمودار کاملا مشخصه که شبکه‌ای که از ReLU استفاده می‌کنه خیلی سریعتر یاد می‌گیره loss توی ReLU توی همون iteration های اول با شیب تند پایین میاد و بعدش هم به‌آرومی به سمت مقدار خیلی کوچیکی همگرا میشه. یه مقدار نوسان اولش دیده می‌شه، ولی در کل روند یادگیری سریع و مؤثره.

در مقابل، Sigmoid خیلی کندتر عمل میکنه loss تقریبا توی کل training بالا می‌مونه و کاهش خیلی آهسته‌ای داره. دلیل اصلی این رفتار هم مشکل vanishing gradient هست. توی Sigmoid وقتی ورودی‌ها بزرگ یا خیلی منفی می‌شن، خروجی تابع به 0 یا 1 نزدیک میشه و مشتقش تقریبا صفر می‌شه. این باعث می‌شه گرادیان‌ها وقتی از لایه‌های آخر به لایه‌های اولیه برمی‌گردن، مدام کوچیک‌تر بشن و عملا وزن‌های لایه‌های ابتدایی درست آپدیت نشن.

به‌خاطر همین، شبکه‌ای که Sigmoid داره سختتر یاد می‌گیره و خیلی دیر همگرا میشه. در مقابل، ReLU توی ناحیه‌ی مثبت مشتق ثابت و غیرصفر داره، بنابراین گرادیان‌ها بهتر منتقل می‌شن و شبکه سریع‌تر و عمیق‌تر یاد می‌گیره.

در مجموع می‌شه گفت ReLU هم سریع‌تر همگرا می‌شه و هم از مشکل vanishing gradient تا حد زیادی جلوگیری می‌کنه، برای همین توی شبکه‌های عمیق انتخاب بهتری نسبت به Sigmoid هست.

3.3) وقتی learning rate خیلی بزرگ (1.0) انتخاب شده، مقدار loss نوسان زیادی داره و عملا درست همگرا نمیشه. دلیلش اینه که هر آپدیت وزن‌ها خیلی بزرگه و مدل مدام از روی نقطه‌ی بهینه رد میشه (overshooting)، برای همین loss بالا می‌مونه و ثبات نداره.

برای learning rate متوسط(0.1) ، رفتار خیلی بهتره. loss توی iteration های اول سریع پایین میاد و بعدش هم با نوسان کم به مقدار کوچکی همگرا می‌شه. این حالت یه تعادل خوب بین سرعت یادگیری و پایداری ایجاد کرده و عملا بهترین نتیجه رو بین این سه تا میده.

در مقابل، وقتی learning rate خیلی کوچک (0.001) باشه، مدل خیلی آروم یاد می‌گیره loss تقریبا یکنواخت و بدون نوسانه، ولی کاهش اون خیلی کنده و بعد از تعداد زیادی iteration هم هنوز به مقدار مناسبی نرسیده.

در مجموع می‌شه گفت learning rate خیلی بزرگ باعث overshooting و ناپایداری می‌شه، learning rate خیلی کوچک باعث همگرایی خیلی کند، و یه learning rate متوسط معمولا بهترین بالانس بین سرعت و پایداری رو ایجاد می‌کنه.

3.4)

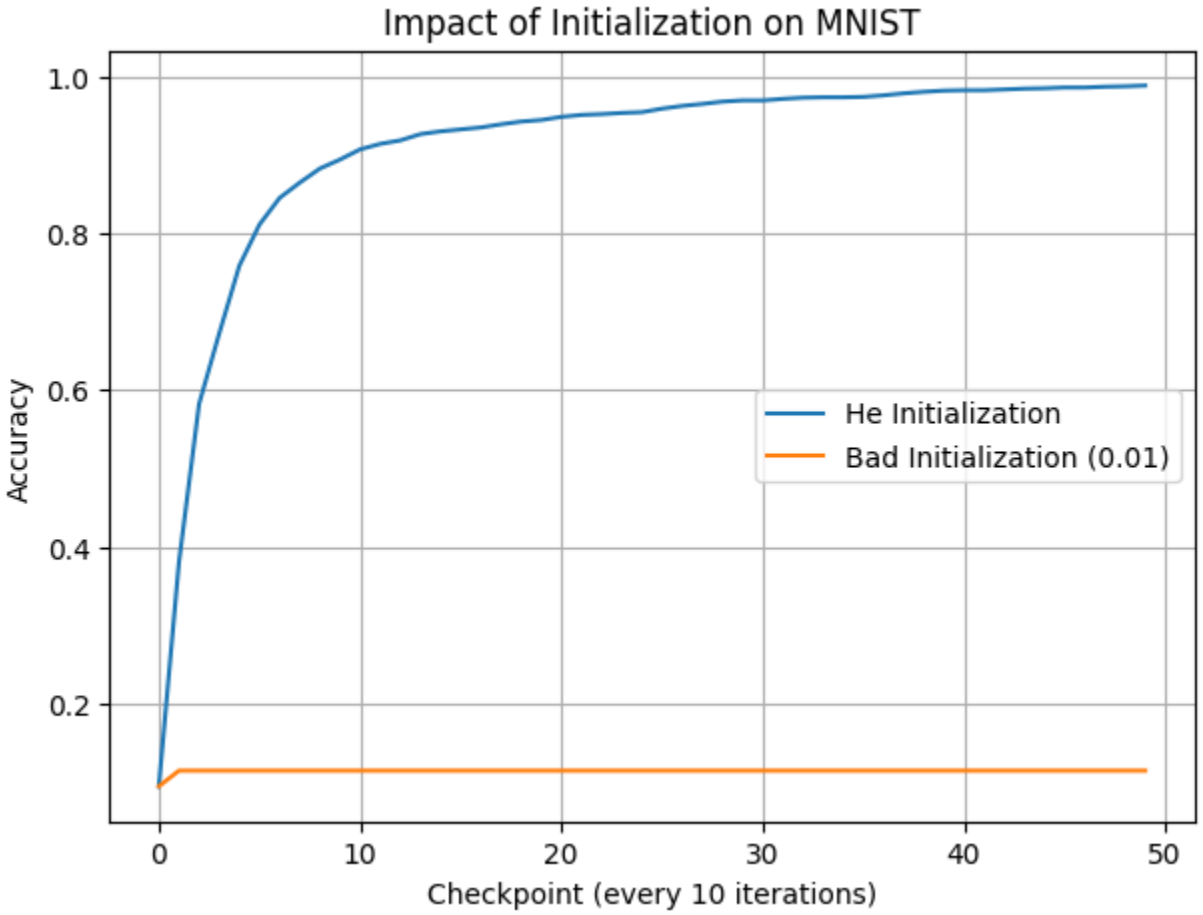
```
Calculating accuracies...
--- Analysis 3.4 Results ---
WITH Regularization (lambda=0.1):
  Training Accuracy: 96.93%
  Test Accuracy:     95.92%
  Gap:               1.01%

WITHOUT Regularization (lambda=0):
  Training Accuracy: 96.93%
  Test Accuracy:     95.93%
  Gap:               1.00%
```

از نظر دقت تست، هر دو مدل عملکرد تقریبا یکسانی دارند. مدل بدون Regularization دقت تست کمی بالاتر داره، اما این تفاوت بسیار ناچیز است.

فاصله بین دقت train و تست در هر دو مدل حدود 1 درصد است که مقدار کمی محسوب می‌شود. این موضوع نشان میدهد که مدل بدون Regularization نشانه‌ای از overfitting ندارد و Regularization با $\lambda = 0.1$ نیز تأثیر محسوسی بر بهبود مدل نداشته است.

(3.5



مطابق نمودار تفاوت بسیار زیادی بین مقدار دهی اولیه he با مقدار دهی ضغیف وجود دارد

مدل با مقداردهی اولیه ضعیف (وزن‌های بسیار کوچک) تقریبا هیچ پیشرفتی در طول آموزش ندارد. دقت این مدل در تمام مراحل آموزش در محدوده‌ی 10 درصد باقی می‌ماند که تقریبا معادل حدس تصادفی میباشد.

طبق vanishing gradient زمانی که وزن‌ها با مقادیر بسیار کوچک مقداردهی می‌شوند خروجی فعال‌سازی‌ها در هر لایه بسیار کوچک می‌شود، گرادیان‌ها هنگام Backpropagation به تدریج کوچکتر می‌شوند و لایه‌های ابتدایی شبکه گرادیان‌هایی نزدیک به صفر دریافت می‌کنند.

در نتیجه، وزن‌ها به‌درستی به‌روزرسانی نمی‌شوند و فرآیند یادگیری عملا متوقف می‌شود.